

REPLICATION.md

Replicating Package-Hallucination Measurement Through a Provider-Agnostic HTTP Client

A partial replication of Spracklen et al., USENIX Security '25

Technical report accompanying the API/Ollama extension in this repository. Version 4 — 2026-08-13.

Version	Date	Change
1	2026-08-11	Initial report: single CodeLlama 7B run at n=100 per dataset.
2	2026-08-12	Two-model replication, response-cap finding, gpt-oss:20b measurement.
3	2026-08-12	Response to an independent audit (codex-experiment.md): DeepSeek reclassified as parser-confounded; the cap "tail" arithmetic withdrawn and re-measured on valid pairs; the run-to-run variance claim withdrawn (nested samples); prompt-cluster bootstrap intervals throughout; gpt-oss decoupled from the paper's scale; registry staleness quantified. Every audit claim was re-verified against the raw artifacts before adoption — see AUDIT_RESPONSE.md .
3.1	2026-08-12	Incorporates the audit's forensic addendum, verified exactly: the audit retracted its family-parser re-score (its corrected figures match ours), and the DeepSeek cap effect is reattributed to parser contamination of format-drifted responses (§5.3). Ordering evidence revised — the CodeLlama positional gradient survives cleaning; the DeepSeek tail measurement does not. CodeLlama and gpt-oss runs verified drift-free, so their headline numbers stand.
4	2026-08-13	Adds Part II — Campaign 4 : the preregistered two-track campaign (PREREGISTRATION_v4.md) with a validated grammar-aware parser, a controlled cap diagnostic, properly powered Track A replication (2 models × 3 seeds), and a six-model modern benchmark under dual registries. Sections 1–8 below stand as the Part I historical record, with superseded-by notes where Campaign 4 settled a question. Results artifact: campaign4_results.json ; revised once after a second independent review (prereg Amendment 5).

Abstract

Spracklen et al. measured package hallucination across 16 code-generating LLMs by loading model weights locally with HuggingFace `transformers`. Extending that study to models available only behind

hosted APIs requires re-implementing the generation stage as HTTP calls. A re-implementation is only useful if it measures the same thing as the original, so before applying it to new models we tested it against the original.

We reconstruct the paper's hallucination-rate metric from its own appendix and verify the reconstruction arithmetically: our transcription of Tables 7 and 8 sums to 440,445 hallucinated packages out of 2,235,642 — exactly the headline figures reported in §5.1 — with per-language means of 15.87% and 21.38% against the stated 15.8% and 21.3%. We then re-implement only the generation stage.

Replicating CodeLlama 7B gives approximate agreement: 23.84% against a published 26.12% (prompt-cluster 95% CI [22.18%, 25.47%], excluding the published value), with the same response parser as the original on both sides. A DeepSeek 6.7B run initially presented as a second replication is **reclassified**: an independent audit found it was scored with the generic parser where the original pipeline selects a DeepSeek-specific one. The audit's first re-score suggested a dramatic parser divergence; it later retracted those figures as a computation error, and its corrected numbers match ours exactly (generic 13.41%/24.89% versus family-parser 12.69%/20.76% at the two caps) — independent convergence on a modest parser delta.

The real instrument finding, from the audit's forensic addendum and verified here number for number, is an **interaction between the response cap and permissive parsing**. Raising the cap moves DeepSeek's measured rate by +11.5 pp, but the shift is concentrated in one query type (+17.6 pp versus +2.8 pp) and is predominantly **evaluator contamination**: at the larger budget DeepSeek drifts out of the requested comma-list grammar (code fences in 338/800 long responses versus 132/800 short; numbered lists 208 versus 130), and the comma-splitting parser scores code fragments, arguments, and prose as hallucinated packages. Excluding format-failed responses, the cap contrast on that query collapses (15.09% versus 14.03%), and two single responses account for 21% of the raised-cap "hallucinations". An unanchored normalization regex compounds it, corrupting even valid names ("12. requests" → "1requests" , scored hallucinated). A genuine truncation-suppression effect may exist — on the clean subset the literal tail still runs 5.9% against a 1.0% head, and within clean CodeLlama responses the hallucination rate rises from 21% at position 1 to ~35% by position 5 — but it is small where measurable and not causally identified. The same cap fails completely on a reasoning model: on 15/15 queries gpt-oss:20b spent the entire 64-token budget on hidden reasoning and returned empty content — zero packages extracted — while a non-reasoning control was unaffected.

Measured under a modernized protocol (raised caps) rather than the paper's, gpt-oss:20b produces 89 flagged recommendations out of 3,071 (**2.90%**, cluster CI [2.18%, 3.64%]); re-checking its flagged names against the live registry reclassifies two, giving 2.83%. Its 800 package-query responses contain zero code fences or numbered lists, so this number is unaffected by the contamination mechanism; the CodeLlama replication is likewise verified drift-free (8 of 3,200 responses flagged, clean-subset rate identical). The gpt-oss result is not directly comparable to the paper's numbers and we make no ratio claims against them.

Part II executes the redesign all of the above demanded, as a preregistered two-track campaign with a validated grammar-aware parser and dual registries. Its headlines: the paper's result **replicates** at proper

power — ordering at every seed, ~1.5–2 pp low, regeneration noise only 0.5 pp — though exact DeepSeek replication fails under the historical parser; the claimed cap effect **vanishes entirely** under deterministic, contamination-free measurement; the original methodology could not see stdlib module-confusion because squatters had registered those names (since purged — 8,961 deletions from the 2024 registry); and across six modern models, frontier systems measure **1.3–2.2%** unregistered-PyPI recommendations while two models exhibit a new failure shape — near-clean typical answers with rare degenerate enumerations of hundreds of names, making prompt-level risk and occurrence rate disagree by design.

Artificial Intelligence Disclosure

The code and analysis of this work was made entirely by Anthropic Opus 5, Fable 5, and Open AI Codex 5.6 Sol, under the direction of Jacob Anderson (jwa@beyond-ordinary.com, @bojake on github). All work is derived from the original work at <https://github.com/Spracks/PackageHallucination>.

Part I — The replication, and what it revealed about the instrument (v1–v3.1 record)

Sections 1–8 are the historical record of the replication effort. Where Campaign 4 (Part II) settled a question these sections left open, a superseded-by note says so.

1. Motivation

The original pipeline requires a local copy of each model's weights. This is a hard limit on what can be studied: the model list is fixed at what fits on the researcher's hardware and at what was publicly downloadable in 2024. Every widely used commercial model released since is reachable only over HTTP.

Replacing `model.generate()` with an HTTP request is mechanically simple. The risk is not that it fails loudly but that it succeeds while measuring something subtly different — a changed prompt, a dropped sampling parameter, a response parser that no longer matches the model's output format. Any of these shifts the measured rate without producing an error. The purpose of this report is to establish what the re-implementation does and does not measure, on cases where published answers exist. Version 3 exists because an independent audit found that versions 1 and 2 overclaimed in specific ways; its findings were verified against the raw artifacts and are incorporated throughout.

2. The original measurement

2.1 Pipeline

For each model and language, the study runs four prompt datasets: two derived from Stack Overflow questions (most popular all-time, and from the preceding year) and two generated by an LLM from popular package descriptions on the same two time bases. Each prompt yields one code sample, and package names are extracted by three heuristics (§4.3 of the paper):

- 1. Parse `pip install` / `npm install` commands out of the generated code.
- 2. Ask the model which packages its own generated code requires.
- 3. Ask the model which packages would solve the original prompt.

Extracted names are normalized and compared against a master list of PyPI/npm package names snapshotted 2024-01-10. Any name absent from that list is a hallucination.

2.2 The metric, and its verification

§5.1 defines the package hallucination rate as "a simple ratio of the number of hallucinated packages to the total number of recommended packages." That leaves open which of the three heuristics enter the denominator. Tables 7 and 8 (Appendix E) settle it: they report each model's rate broken out by LLM-generated prompts, Stack Overflow prompts, and `pip / npm install`, alongside a total. For all 30 model×language rows, the three component numerators sum exactly to the total numerator, and likewise for denominators. For GPT-4 Turbo on Python: $1,518 + 1,169 + 52 = 2,739$, and $46,204 + 28,728 + 1,381 = 76,313$.

The metric is therefore a plain pooled ratio over all three heuristics and all four datasets, with no per-query or per-dataset weighting:

```
rate = (hallucinated from query 1 + query 2 + install commands)
      / (all packages recommended by those three heuristics)
```

We transcribed both tables into `Baselines/paper_appendix_e.csv` and verified the transcription three ways:

Check	Result
Each row's components reproduce its published percentage	30/30 within 0.005 pp
Sum over all rows	$440,445 / 2,235,642 = 19.70\%$ — the paper's headline figures exactly
Per-language means	Python 15.87%, JavaScript 21.38% (paper: 15.8%, 21.3%)

Three cells were damaged in the PDF text layer; each was reconstructed from the additive identity and independently confirmed by the percentage printed beside it (`Baselines/README.md`). The independent audit re-verified all of the above and it stands.

We note inconsistencies in the repository's plotting data, documented in `Baselines/README.md`: `figure_2.csv` disagrees with Tables 7 and 8 for every model (the paper's own prose quotes the appendix values), `figure_14.csv`'s column order is the reverse of its plot's axis labels, and `figure_9.csv` sums to 76,395 observations where the paper's prose says 76,489, with the prose's "10,263 at distance 1 or 2" matching the plot data only if distance-0 entries are included. We use the appendix tables as the baseline.

3. Implementation

The re-implementation replaces the generation stage only.

Unchanged. Prompt text is copied verbatim **from the released artifact** (we have not independently diffed the artifact's prompts against the camera-ready paper's Appendix B), including two quirks that would otherwise invite silent "fixes": code generation folds the system instruction into the *user* turn with no separator and no system role, and the language string is interpolated raw, so JavaScript runs say "Javascript". Sampling parameters are the paper's (Table 6): temperature 0.7 for code generation and 0.01 for package queries, top-p 0.9, top-k 20, response caps 2048 and 64. Detection, normalization, master-list comparison, and aggregation call the original `package_detection.py`, `custom_parse_*.py` and `aggregate_results.py` unmodified.

Necessarily different. Generation is an HTTP POST rather than a local forward pass. Where the original loaded GPTQ-quantized weights, a local run now goes through Ollama's native `/api/chat`, which accepts all three sampling parameters. Hosted providers vary: OpenAI and xAI have no `top_k`, so it cannot be sent. Every such difference is recorded per run in a manifest, which as of this version also records the served model's immutable identity (Ollama digest, quantization, server version) and the model id echoed in responses.

Response parsing — one correct, one not. The original selects a hand-written parser per model family. For CodeLlama on Python, `get_pre_post_info()` applies no family-specific processing, which is exactly what this runner's default does — parsing for the CodeLlama replication is identical on both sides. For DeepSeek the original routes through a DeepSeek-specific pre/post parser, and **our runs did not** — they used the generic default. We failed to disclose this in versions 1–2; the independent audit caught it, and §5.2 shows the consequences are large.

4. Method

Models. CodeLlama 7B (paper: 26.12%, the highest Python rate in the study) and DeepSeek 6.7B (paper: 16.61%), served by Ollama as `codellama:7b-instruct` and `deepseek-coder:6.7b-instruct` — instruction-tuned variants, since the original applies a chat template. Separately, `gpt-oss:20b`, a 2026 open-weight reasoning model outside the original study.

Settings. For the replications, the paper's exact values: caps 2048/64, temperature 0.7/0.01, top-p 0.9, top-k 20. Manifests record `deviations_from_paper: none` and `request_adjustments: none` — full sampling fidelity. gpt-oss required raised caps for the reason established in §5.4.

Sampling design. A full run is roughly 19,200 code samples per model per language. We draw random samples of 100–400 prompts per dataset (seed 0). Random sampling matters: the Stack Overflow datasets are ordered by question popularity, so a prefix is a biased subset. One consequence discovered by the audit: because the subsets are seeded identically, the n=100 prompt set is **nested inside** the n=400 set, which we verified — useful for paired regeneration comparisons, fatal for treating the two runs as independent (§5.5).

Statistics. Package recommendations cluster within prompts, so package-level binomial intervals understate uncertainty. All intervals in this report are **stratified prompt-cluster bootstraps** (prompts resampled with replacement within each of the four datasets, 2,000 replicates), computed by `compare_to_paper.py`, which prints them by default. Our intervals agree with the audit's independent bootstrap to within 0.2 pp everywhere. Generation is not seed-controlled at the model, so repeat-generation variance is a separate, mostly unmeasured component (§5.5).

5. Results

5.1 CodeLlama 7B: approximate replication, biased low

Run	Packages	Rate	Cluster 95% CI	Paper	CI contains paper?
n=400/dataset	6,258	23.84%	[22.18, 25.47]	26.12%	no
n=100/dataset (nested subset)	1,464	26.43%	[22.91, 29.89]	26.12%	yes

Zero failed requests. The better-powered run sits 2.28 pp below the published value with an interval excluding it: **approximate agreement with a low bias, not exact reproduction**. Both runs used the same parser the original used for this model, so parser choice is not a confound here, and format-drift contamination is negligible for this model (8 of 3,200 package responses contain a code fence or numbered list; the clean-subset rate is identical at 23.96%). With the truncation-suppression account of v2–v3 withdrawn (§5.3), the 2.3 pp gap is at present **unexplained**; checkpoint, quantization, and chat-template differences (§6) are the remaining candidates.

Sub-rate composition still differs from the paper in ways we cannot fully explain (LLM-prompt sub-rate low at 17.50% vs 21.51%; Stack Overflow close at 31.57% vs 32.53%), and the run extracts 3.9 packages per code sample against the paper's 5.5. The headline number is closer than the parts.

5.2 The DeepSeek result is parser-conditional, and is reclassified

The audit found our DeepSeek runs were scored with the generic parser while the original pipeline routes DeepSeek through a family-specific pre/post parser (`(True, True, "DeepSeek")`) from

`get_pre_post_info()`). Both run manifests confirm it. Versions 1–2 of this report presented 13.41% as a replication of the paper's 16.61%; that presentation was wrong, and the claim is withdrawn.

Re-scoring the **identical stored response text** under both parser paths:

Parser interpretation	cap 64	cap 2048
Generic default (as originally run)	13.41% (411/3,064)	24.89% (1,063/4,270)
Family parser, repo's own code path ¹	12.69% (466/3,672)	20.76% (1,208/5,820)

¹ `package_detection.detect_packages(..., overrides=(True, True, "DeepSeek"))` — reproduce with `verify_audit_findings.py`.

The audit initially reported the family parser giving ~51% at both caps. It has since **retracted those figures** — they came from applying the parser a second time to list-valued fields already serialized in `*_results.csv` rather than to the raw responses — and its corrected numbers reproduce ours exactly. Two independent implementations now converge: family-versus-generic parsing shifts the DeepSeek result modestly (and narrows the cap contrast from 11.5 to 8.1 pp), rather than swinging it by 4×.

The consequential parser finding is elsewhere (§5.3): the generic parser's permissive comma splitting converts *format drift* into counted hallucinations, and its unanchored numbered-list normalization (`\d\.` applied anywhere in the string) corrupts even valid names — `"12. requests"` becomes `"1requests"` and is scored as a hallucination. Parser behaviour therefore still has to be validated against manually labeled responses before any cross-implementation comparison, and the original artifact remains only **partially portable**: family parsers encode format assumptions about 2024 checkpoint output that a different serving stack need not honour. CodeLlama measures above DeepSeek under both parser paths, matching the paper's ordering.

5.3 The cap and the parser interact: the measured cap effect is mostly an evaluator artifact

Settled by Campaign 4 (Part II §C): the controlled diagnostic — temperature 0, seeded, counterbalanced caps, grammar-aware parser — finds **no cap effect at all** on either DeepSeek generation. The analysis below stands as the discovery path.

The original caps package-query responses at 64 tokens (Table 6). DeepSeek hit that cap on 34.5% of package queries against CodeLlama's 2.4%. Re-querying the same generated code with only the cap changed:

Package-query cap	Rate (generic parser)	Cluster 95% CI	Responses hitting cap
64 (paper's setting)	13.41%	[11.60, 15.40]	828/2,400
2048	24.89%	[20.46, 29.91]	1/2,400

Versions 2–3 attributed this +11.5 pp shift to truncation suppressing genuinely hallucinated tails. The audit's forensic addendum identified the dominant mechanism as something else, and we verified its decomposition exactly:

The shift is concentrated where format compliance breaks down. Query 1 ("packages required to run this code") moves +2.78 pp (11.18% → 13.96%); Query 2 ("packages useful for this problem") moves **+17.59 pp** (15.88% → 33.47%). At the larger budget, Query 2 responses drift out of the requested comma-list grammar — code fences appear in 338/800 long responses versus 132/800 short, numbered lists in 208 versus 130 — and the permissive comma-splitter then scores function arguments, string literals, and prose fragments as hallucinated packages. Two single responses contribute 171 of the 805 raised-cap Query 2 "hallucinations" (21%). The unanchored normalization regex adds corruption of its own: "33. docker-container-run" → "3docker-container-run", and "12. requests" → "1requests" — a **real** package scored as a hallucination.

Excluding format-failed responses collapses the effect. On Query 2 responses with no code fence and no numbered list: 15.09% at cap 64 versus 14.03% at cap 2048. (Diagnostic, not a replacement estimate — format compliance is itself cap-dependent, so the conditioning can introduce selection bias. At the paper's own 64-token setting the contamination is modest: 15.88% versus 15.09% clean.)

What survives of the ordering hypothesis. The v2 "discarded tail is 54% hallucinated" arithmetic remains withdrawn (of 1,600 pairs: 628 identical, 242 strict extensions, 730 divergent — verified). Re-measuring on clean data only: the CodeLlama within-response gradient **survives** (21.2% at position 1 rising to 35.3% at position 5+, on responses with no drift markers — CodeLlama barely drifts at all), while the DeepSeek strict-prefix tail measurement **collapses** from 38.8% to 5.88% (3/51) against a 1.03% head once format-failed long responses are excluded. Valid-first ordering is real for at least CodeLlama; as an explanation of large cap contrasts it is not supported.

Conclusions, revised. The 64-token cap is a real instrument hazard in **two directions**: it silently zeroes reasoning models (§5.4), and *raising* it without a grammar-aware parser manufactures false hallucinations for format-drifting models — which is what our +11.5 pp mostly was. A genuine truncation-suppression effect may exist (the +2.78 pp Query 1 residual; the tiny clean-tail excess), but the present runs cannot identify it causally: 45.6% of paired responses diverge before the truncation point even at temperature 0.01, and the audit's seed probe found explicit seeds insufficient for paired trajectories in this Ollama configuration. v2–v3's inference that the paper's published rates are underestimates is **withdrawn**. The comparability warning changes shape rather than disappearing: a fixed cap plus a permissive parser confounds cross-model comparison through *format compliance*, not verbosity per se — and malformed-response rate should be reported as an outcome in its own right, exactly as the audit recommends. The definitive experiment remains offline truncation of captured token sequences with a grammar-aware, manually validated parser.

5.4 The same cap silently zeroes a reasoning model

Re-run with committed provenance (input SHA-256, model digests, server version — Experiments/cap_experiment_summary.json ; raw responses retained locally, see §8):

Model	Cap	Empty responses	Hit cap	Packages extracted
gpt-oss:20b (reasoning)	64	15/15	15/15	0
gpt-oss:20b (reasoning)	2048	0/15	0/15	24
codellama:7b-instruct (control)	64	0/15	0/15	23
codellama:7b-instruct (control)	2048	0/15	0/15	23

The control is indifferent to the cap; the reasoning model spends the 64-token budget on hidden reasoning and returns nothing. Applied verbatim, the 2024 protocol scores a reasoning model as recommending no packages at all — total, silent measurement failure, reported by the pipeline as a well-formed "no packages" outcome. Any application of this methodology to a reasoning model must raise the cap and say so; runs here record cap hits per phase in the manifest.

5.5 What the two CodeLlama runs do and do not show about variance

Settled by Campaign 4 (Part II §D): three independent generation seeds per model on a fixed prompt set measure the regeneration spread at **0.5–0.6 pp** — small, as the single paired observation below suggested.

Version 2 claimed "run-to-run variance is ± 3 pp at best." **Withdrawn.** The audit observed — and we verified — that the n=100 prompt set is nested inside the n=400 set, so the runs were never independent. Partitioning the n=400 run:

Portion	Rate
n=100 run itself	26.43% (387/1,464)
The same 100 prompts, regenerated inside the n=400 run	26.01% (385/1,480)
The 300 added prompts	23.17% (1,107/4,778)

Regeneration moved the repeated prompts by only 0.42 pp (one paired observation — suggestive that generation noise is small, not a measurement of it). The aggregate 2.6 pp shift came from prompt composition. The correct uncertainty statement is the prompt-cluster interval, which for the n=100 run spans ± 3.5 pp — wide enough that its apparent exact agreement with the paper in version 1 was, as the audit put it, an artifact of low power. Estimating generation variance proper needs repeated runs on a fixed prompt set, which we have not done.

5.6 A 2026 open-weight reasoning model under a modernized protocol

Measured with caps raised to 4096/2048 — **not the paper's protocol, and not comparable to its numbers:**

	Value
Hallucination rate (frozen 2024-01-10 registry)	2.90% (89/3,071), cluster CI [2.18, 3.64]

	Value
Re-checked against live PyPI (2026-08-12)	2.83% (87/3,071)
Packages per code sample	7.68 (the two 2024 models here: 3.8–3.9)
Unique hallucinated names	59
Responses hitting the cap	3

For orientation only: the original study's Python rates span 3.59% (GPT-4 Turbo) to 26.12% (CodeLlama 7B). gpt-oss:20b's 2.90% sits below that entire range, but the protocols differ, so we make no ratio or ranking claims against the paper. The contamination mechanism of §5.3 does **not** inflate this number: all 800 of gpt-oss's package-query responses are clean comma lists — zero code fences, zero numbered lists — and the clean-subset rate is identical to the headline. A defensible cross-model statement still requires the two-track design in §7.

Two of its 59 flagged names are now-registered packages: `python-design-patterns` (first upload 2024-10-18) and `pyjpeg` (first upload **2026-06-17** — roughly ten months after the model's release, the temporal shape of the slopsquatting risk the original warns about, though registration by an unrelated party is indistinguishable from coincidence here). The remaining top repeats are two standard-library module names written as installable packages (9× and 6×) and a plausible but non-existent sibling of a real multi-package framework (5×) — the module-vs-package confusion the original discusses in Appendix G, not free invention. All were verified absent from the frozen master list. *(Redacted 2026-09-27: earlier revisions of this report printed those three still-unregistered names here; they were replaced with descriptions under the release policy. The raw run under Tests/ retains them.)*

5.7 Levenshtein proximity: exploratory, and dominated by the reference set

The original's RQ4 reports that only 13.4% of hallucinated names lie within 1–2 edits of a valid package. Measuring our CodeLlama n=400 names (1,207 unique) against two reference sets:

Reference set	Within 2 edits
Full master list (500,498 names)	42.7%
Packages the run itself validly recommended (1,333 names)	12.1%

The full-list figure is inflated by obscure near-collisions (`opencv` → `openav` at distance 1, rather than the plausibly-intended `opencv-python` at 7). The in-use figure lands near the paper's 13.4% — but that reference set is post-hoc (selected after observing the run, sized by it), so, per the audit, proximity to the paper's number is **not** independent validation. We retain one supported conclusion — the statistic swings by 3–4× on a parameter that neither the paper nor prior work states precisely, so any use of it must declare its reference set — and label everything else exploratory. A defensible future design predeclares a dated top-K downloads list. The paper's own Figure 9 data carries internal inconsistencies documented in `Baselines/README.md` .

6. Threats to validity

Permissive parsing converts format drift into counted hallucinations. §5.3: at raised caps, DeepSeek's apparent rate doubles mostly because the comma-splitter scores code and prose fragments from format-failed responses as packages, and the unanchored numbered-list normalization corrupts even valid names ("12. requests" → "1requests"). Family-versus- generic parser choice shifts results modestly by comparison (§5.2). The binding requirement is a grammar-aware parser validated against manually labeled responses, with malformed-response rate reported as an outcome — until then, results are conditional on the parser stated alongside them.

The models are not provably the same checkpoints. GPTQ 4-bit via `transformers` versus Ollama's independently built GGUFs with their own chat templates. Runs now record the served digest, quantization, and server version in the manifest, so future results are at least pinned to *something* immutable; the 2024 originals are not recoverable.

Sampling is not seed-controlled, and our two same-model runs were not independent. §5.5. Prompt-cluster intervals are reported throughout; generation variance is unmeasured beyond one paired observation. The original's published figures are also single stochastic runs with no stated interval.

Registry time. The frozen 2024-01-10 list is correct for replication and wrong as the sole ground truth for 2026 models: names registered since score as hallucinations (quantified for gpt-oss in §5.6 — small there, not small in general). Modern measurements should score against frozen *and* current snapshots; a current lookup still cannot distinguish a legitimate new package from a squatted hallucination.

Response caps. §5.3–5.4. The replications used the paper's caps for comparability, which truncated 34.5% of DeepSeek's package queries; the extension's defaults are higher, which breaks comparability with the paper. There is no cap setting that is simultaneously faithful and neutral; the manifest records the choice.

Inherited noise filters. The original's `false_positive_packages.csv` (3,882 entries) is applied to new models' output unchanged. For new models it may suppress genuine outputs; future work should version and validate it rather than inherit it as a blocklist.

Prompt provenance. Prompts are verbatim from the released artifact; we have not diffed them against the camera-ready appendix.

7. What this does and does not license

It licenses: measuring models the original could not reach, with recorded deviations; ordering-level and band-level statements (a 3% model versus a 15% one) within runs produced by this pipeline under one parser and one cap policy; and CodeLlama-style approximate replication claims where the original parser is the shared default.

It does not license: exact-agreement claims against the paper (our best-powered replication excludes the published value); any DeepSeek fidelity claim pending manually-validated parsing; reading a single

run at finer resolution than its prompt-cluster interval; cross-protocol ratios (gpt-oss versus the paper's table); attributing a cap-contrast or any raised-cap rate increase to model behaviour without first auditing format drift (§5.3); Levenshtein proximity figures without a declared reference set; or treating two cells of a 30-cell grid as validation of the rest.

The audit's recommended next iteration — a preregistered two-track design separating historical artifact replication from a modern-model benchmark, with parser validation against labeled samples, offline cap truncation, fixed-prompt repeated seeds, and dual registry snapshots — is the right structure, and this repository now records identities and per-phase statistics sufficient to support it.

8. Reproduction

```
pip install -r requirements-api.txt
python compare_to_paper.py --verify-baseline    # checks §2.2 with no model needed

ollama pull codellama:7b-instruct
python run_test_api.py ollama:codellama:7b-instruct --language Python \
    --sample 400 --seed 0 --max-code-tokens 2048 --max-package-tokens 64
python compare_to_paper.py Tests/ollama_codellama_7b-instruct_Python # cluster CIs by
    default
python analyze_hallucinations.py Tests/ollama_codellama_7b-instruct_Python # incl.
    positional analysis

python cap_experiment.py                    # §5.4, writes Experiments/cap_experiment_summary.json
python verify_audit_findings.py            # re-checks every §5.2/5.3/5.5 number from raw
    artifacts
```

Run manifests record every parameter transmitted, every deviation from the paper's settings, per-phase cap hits, and the served model's digest. Per-prompt result files and raw responses stay under `Tests/`, which is deliberately not committed: model outputs contain hallucinated package names, which this repository — like the original — does not publish. Auditors regenerate them with the commands above; the committed summaries carry input hashes and digests so regenerated runs are attributable.

Part II — Campaign 4: the preregistered two-track campaign (2026-08-12/13)

Part I ended with a design: separate historical replication from modern benchmarking, validate the parser against labeled data, isolate the cap effect deterministically, treat prompts as clusters, and pin model identities. Campaign 4 executed that design under a frozen preregistration ([PREREGISTRATION_v4.md](#), signed before execution; all deviations logged as amendments). Every number below is in

[campaign4_results.json](#), regenerated once after a second independent review whose findings were verified and adopted (Amendment 5).

A. Instrumentation

Parser v2 accepts only responses conforming to a list grammar; everything else is **malformed** — an outcome, never a source of package names. Validated on a stratified 200-response sample, labels drafted by the assistant and signed after delegated review (Amendment 4): response-status accuracy **98%** (four errors, one root cause — wrappers around an entire comma list), extraction precision **100%**, grammar-valid occurrence recall **97.8%**, end-to-end semantic recall **79.7%** (the deliberate cost of refusing to mine prose). Known limitation: the sample predates the hosted cells, so hosted-model response formats are unrepresented. Frozen v2.1 candidates (response-level wrapper stripping, cap-truncated trailing items, no-packages prose) are documented, not applied.

Primary metric — the unregistered-PyPI recommendation rate: a recommended name absent from **both** the frozen 2024-01-10 registry (500,513 names) and the dated current snapshot (2026-08-12; 869,894 names — the namespace grew 74%), and not a standard-library module. The three qualifiers each earn their place: 8,961 names in the frozen list have since been *deleted* from PyPI — including squat-registrations of stdlib names (`os` , `sys` , `json`), which means the original methodology silently scored stdlib confusion as *valid packages* via squatters, and its Python pipeline (unlike its JavaScript one) had no core-module exclusion. "Unregistered" rather than "invented": degenerate responses often name real apt/npm/system tooling (`gcc` , `python-dev`) — invalid as PyPI recommendations, not fabricated strings. Statistics are prompt-cluster bootstraps stratified by dataset throughout; cross-model comparisons are paired by prompt with Holm correction across all 15 pairs.

B. Track A — the paper's result replicates, with a characterized bias

Paper protocol exactly (caps 2048/64, temperature 0.7/0.01, top-k 20, top-p 0.9), three independent generation seeds per model on one frozen 200-prompt-per-dataset subset:

Model	Paper	Seeds (generic parser)	Mean Δ	Family parser (mean)
CodeLlama 7B	26.12%	24.20 / 24.84 / 24.60	−1.57 pp	(generic is historical)
DeepSeek 6.7B	16.61%	14.86 / 14.37 / 14.62	−1.99 pp	13.43%

Seed spread is 0.5–0.6 pp — regeneration noise is small, settling Part I §5.5. Ordering reproduces at every seed. Under the **historical family parser** — the acceptance condition — all three DeepSeek CIs (≈[11.7, 15.4]) **exclude** the published 16.61%: ordering and broad magnitude reproduce; exact replication does not. The consistent ~1.5–2 pp low bias across both models and all seeds points at checkpoint/quantization/template provenance, which digests now pin going forward but cannot recover for 2024.

C. The cap question, answered

The controlled diagnostic (temperature 0, seeded, one worker, counterbalanced caps {64...2048}, prefix-verified pairing, Parser v2): **no cap effect**. DeepSeek 6.7B pooled rates fluctuate 4.8–6.7% with no monotone trend; Coder-V2 is flat to within 0.6 pp across a 32× cap range; malformed rates are stable across caps. The saga closes: v2's "+11.5 pp cap effect" decomposed into format-drift contamination of a permissive parser (Part I §5.3) plus sampling noise — with drift itself a *temperature* phenomenon, not a budget one. What remains true: a 64-token cap **zeroes reasoning models entirely** (15/15 empty responses), and cap-truncated list tails interact with any parser. The cap's dangers are real; its advertised effect was not.

D. Track B — six modern models, one instrument

Modern protocol (caps 4096/2048, think /reasoning disabled where the model reasons, n=200/dataset — the gpt-oss cell rerun at conforming size after the review caught the original n=100 as an unlogged deviation):

Model	Unregistered rate	Cluster CI	P(prompt ≥ 1)	Unique names	Top-3 share
claude-opus-5	1.31%	[1.04, 1.62]	12.1%	95	10.7%
gpt-5.2-2025-12-11	1.88%	[1.49, 2.31]	12.1%	102	8.1%
gpt-oss:20b	2.15%	[1.76, 2.57]	13.1%	91	5.2%
grok-4.6	8.22%	[1.57, 18.08]	5.5%	532	82.9%
deepseek-coder-v2:16b	10.30%	[8.80, 11.93]	21.0%	196	4.0%
deepseek-v4-flash	17.91%	[5.66, 29.90]	18.1%	1,045	72.4%

Seven of fifteen paired comparisons survive Holm at 0.05: the frontier trio beats both DeepSeek cells decisively (Δ 8–16 pp each), and claude-opus-5 measures below gpt-oss (Δ −0.84 pp, $p=0.003$). claude-opus-5 versus gpt-5.2 is directionally consistent (−0.57 pp) but does **not** survive correction ($p\approx0.066$) — the first assembly reported it significant from an unstratified 2,000-replicate bootstrap, and the review's challenge was verified and adopted.

The two preregistered outcomes disagree about grok-4.6, and that disagreement is the finding. It has the *lowest* prompt-level risk in the cohort (5.5% of prompts yield any unregistered name) and one of the highest occurrence rates — because three responses carry 83% of its total, in degenerate enumerations of hundreds of names (v4-flash: same shape, 72% / 1,045 unique names). A typical grok or v4-flash answer is fine; the tail free- associates entire package inventories. The failure mode of 2024 — steady, diffuse invention (CodeLlama at 26% spread across a fifth of prompts) — has bifurcated by 2026 into near-clean typical behaviour with rare catastrophic tails in some models, and the two outcomes must be read together. The concentration also destroys grok's statistical resolution (CI [1.6, 18.1]): three responses deny it a rank in either direction against the frontier trio.

Where the 2024→2026 comparison is licensed at all, it is at the band level: the paper's best model measured 3.59% under an instrument that both manufactured hallucinations (parser contamination) and hid them (stdlib squats, truncation); current frontier models measure 1.3–2.2% under a validated parser and dual registries. The stdlib category — 10–20% of recommendations in every cell, peaking at coder-v2 — is module-versus-package confusion the original could not see at all.

E. Limitations carried forward

Parser validation does not cover hosted-model formats (§A). The unregistered rate carries a small upward bias from cap-truncated grammar-valid fragments (the `two` case) pending the v2.1 rule. Six models is not a cohort census; single runs per Track B cell mean between-run variance is bounded only by Track A's seed evidence. The 2024 checkpoints remain unrecoverable, so the Track A bias is characterized, not explained. And two AI systems (this assistant and its reviewer) performed both the analysis and its checks; the maintainer's sign-offs are the human control points, and every verification is committed as runnable code rather than asserted.

References

1. J. Spracklen, R. Wijewickrama, A H M N. Sakib, A. Maiti, B. Viswanath, M. Jadliwala. *We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs*. USENIX Security Symposium, 2025. arXiv:2406.10279v3.
 2. OpenAI Codex. *Independent Audit of the Package-Hallucination Replication and Extension*. [codex-experiment.md](#), 2026-08-12 — and the maintainers' verification and response, [AUDIT_RESPONSE.md](#). Codex additionally performed the delegated Parser v2 label sign-off and the Phase 5 assembly review (Campaign 4, Amendments 4–5).
 3. Campaign 4 artifacts — [PREREGISTRATION_v4.md](#), [campaign4_results.json](#), [parser_v2_validation.json](#), [cap_diagnostic_summary.json](#).
 4. This repository — [EXTENSION_NOTES.md](#) for design decisions and the running verification record; [Baselines/README.md](#) for the baseline transcription and plot-data discrepancies.
-

Provenance: versions 1–2 of this report, the extension under test, and the baseline transcription were produced by Claude (Opus 5) running in Claude Code, directed interactively by the repository maintainer. Versions 3–4, Campaign 4's design, execution, and analysis were produced the same way by Claude (Fable 5). OpenAI Codex provided two independent audits, the delegated label sign-off, and the Phase 5 assembly review; every externally-sourced claim adopted into this report was first re-verified against the raw artifacts, and every verification is committed as runnable code. The maintainer's preregistration sign-off and delegations are the human control points. The original study, its pipeline, and its detection code are the work of the paper's authors.